HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA ĐÀO TẠO SAU ĐẠI HỌC

BÁO CÁO BÀI TẬP LỚN

MÔN DỮ LIỆU LỚN

ĐỀ TÀI

VERIFIER-GUIDED REPAIR:

SỬA CHỨNG MINH LEAN 4 CÓ KIỂM CHỨNG DƯỚI RÀNG BUỘC CHI PHÍ VÀ ĐỘ TRỄ

Giảng viên: **TS. Nguyễn Kiều Linh**

Lớp: **M25CQHT02-B**

Nhóm: **06**

Học viên:

B25CHHT112
Đào Văn Tâm

B25CHHT088
Trần Quang Đức Dũng

B25CHHT117
Lâm Thành Trung

B25CHHT119
Nguyễn Xuân Tùng

B25CHHT125
Khamsing OUTHAIHUENG

B25CHHT124
Mekdala Nounou

MỤC LỤC

TÓM TẮT **4**
CHƯƠNG 1. GIỚI THIỆU **4**
1.1. Bối cảnh và nút thắt dữ liệu lớn 4
1.2. Câu hỏi nghiên cứu 5
1.3. Đóng góp 5
CHƯƠNG 2. CÔNG TRÌNH LIÊN QUAN **5**
2.1. Môi trường và dữ liệu chứng minh 5
2.2. Sinh và sửa chứng minh bằng LLM 6
2.3. Khoảng trống 6
CHƯƠNG 3. DỮ LIỆU VÀ THU NHẬN **6**
3.1. Nguồn và giấy phép 6
3.2. Định lý gốc và lỗi tự nhiên 6
3.3. Sinh lỗi tổng hợp 7
3.4. Chia tập và chất lượng nhãn 7
3.5. Hạn chế dữ liệu 8
CHƯƠNG 4. PHƯƠNG PHÁP VÀ KIẾN TRÚC **8**
4.1. Tổng quan 8
4.2. Bộ kiểm chứng REPL thường trú 8
4.3. Taxonomy và bộ phân loại 8
4.4. Luật sửa và sửa bằng LLM 9
4.5. Vòng lặp có giới hạn và kiểm toán 9
4.6. Lập lịch, chi phí và độ phức tạp 10
CHƯƠNG 5. THIẾT LẬP THỰC NGHIỆM **10**
5.1. Môi trường 10
5.2. Chiến lược và ma trận thí nghiệm 10
5.3. Chỉ số và tái lập 11
CHƯƠNG 6. KẾT QUẢ **11**
6.1. Khởi động nguội là nút thắt đầu tiên 11
6.2. RQ1: thông báo lỗi có cấu trúc 12
6.3. RQ2: định tuyến, ngân sách thử lại và mô hình 13
6.4. RQ3: khả năng mở rộng 14
CHƯƠNG 7. PHÂN TÍCH LỖI VÀ MỐI ĐE DOẠ TÍNH HỢP LỆ **15**
7.1. Vì sao vòng lặp dừng và vì sao bản sửa bị loại 15
7.2. Độ chính xác từng luật 16
7.3. Theo nhóm lỗi, nguồn lỗi và mô hình 16
7.4. Mối đe doạ tính hợp lệ 17
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN **17**
TÀI LIỆU THAM KHẢO **18**
PHỤ LỤC. SỬ DỤNG AI, PHÂN CÔNG VÀ TÁI LẬP **19**

DANH MỤC HÌNH VÀ BẢNG

Bảng 3.1: Các toán tử sinh lỗi tổng hợp. 7
Bảng 4.1: Taxonomy lỗi v0.2..... 9
Bảng 5.1: Ma trận thí nghiệm; lần lặp 2 và 3 của cấu hình LLM dùng bộ đệm câu trả lời..... 11
Bảng 6.1: RQ1 trên 284 mẫu hold-out, ba lần lặp, Gemini 3.5 Flash-Lite..... 12
Hình 6: 1. Tỷ lệ sửa thành công theo nhóm lỗi và chiến lược trên hold-out 1K.... 12
Bảng 6.2: Ablation trên hold-out 1K, trung vị hai đến ba lần lặp, chênh lệch giữa các lần không quá 0,7 điểm..... 13
Hình 6: 2. Chi phí LLM cho 1.000 mẫu và tỷ lệ sửa thành công; đường chấm là biên Pareto..... 14
Bảng 6.3: Chỉ luật k=1 trên toàn bộ mỗi mức, ba lần lặp, chênh lệch thông lượng không quá 4%. 14
Hình 6: 3. Tăng tốc thông lượng theo số worker ở ba mức dữ liệu so với đường tuyến tính lý tưởng. 15
Bảng 7.1: Lý do dừng ở lần lặp đầu; cơ chế khử trùng lặp cắt 4,6% mẫu khỏi các lần thử vô ích..... 15
Bảng 7.2: Độ chính xác của luật, gộp chỉ luật và lai trên hold-out 1K. 16

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Tiếng Anh	Ý nghĩa
LLM	Large Language Model	Mô hình ngôn ngữ lớn
REPL	Read–Eval–Print Loop	Tiến trình Lean thường trú nhận lệnh kiểm chứng
RQ	Research Question	Câu hỏi nghiên cứu
P50 / P95	50th / 95th percentile	Trung vị và phân vị 95, đo độ trễ đuôi
USS / RSS	Unique / Resident Set Size	Bộ nhớ riêng / bộ nhớ thường trú của tiến trình
k	Attempt budget	Số lần thử sửa tối đa mỗi mẫu
SYN_01 đến TAC_07	Error taxonomy codes	Mã nhóm lỗi trong taxonomy v0.2

TÓM TẮT

Các pipeline tự động hình thức hoá toán học sinh ra hàng chục nghìn chứng minh Lean 4, phần lớn không biên dịch được. Sửa chúng ở quy mô lớn là bài toán dữ liệu lớn: mỗi lần kiểm chứng nguội phải nạp thư viện Mathlib khoảng 6 GB, hàng đợi lỗi tăng theo quy mô, và mỗi lần thử lại bằng mô hình ngôn ngữ lớn tốn token và thời gian. Nhóm xây dựng hệ thống Verifier-Guided Repair gồm bộ kiểm chứng Lean REPL thường trú chạy trong pool worker, bộ phân loại lỗi theo taxonomy 7 nhóm lấy từ thông báo thật của Lean 4.35, bộ định tuyến sang luật sửa hoặc LLM có tiêm chẩn đoán, vòng lặp sửa và kiểm chứng có giới hạn, và bộ kiểm toán ngữ nghĩa chống sửa bằng cách đổi đề. Trên tập 17.031 lỗi có nhãn sinh từ Lean, REPL thường trú giảm thời gian kiểm chứng từ 63 đến 107 giây mỗi tệp xuống 3 ms mỗi định lý. Tiêm chẩn đoán nâng tỷ lệ sửa thành công đã kiểm toán từ 31,7% lên 53,9% và giảm 45% chi phí; ba lần thử đạt 65,1%. Ở mức 10.000 ứng viên, thông lượng tăng 1,76 lần với 2 worker nhưng chỉ 1,88 lần với 4 worker do nghẽn CPU. Toàn bộ thực nghiệm tốn 2,97 USD phí API.

Từ khoá: Lean 4, Mathlib, sửa chứng minh tự động, LLM, kiểm chứng hình thức, worker pool, độ trễ đuôi.

CHƯƠNG 1. GIỚI THIỆU

1.1. Bối cảnh và nút thắt dữ liệu lớn

Trong chuỗi AI4Math của lớp, Nhóm 5 dịch đề toán sang Lean 4 theo lô, Nhóm 6 sửa các ứng viên lỗi, Nhóm 7 và Nhóm 8 dùng chứng minh đã kiểm chứng. Phần lớn ứng viên hỏng vì sai cú pháp, thiếu import, tên lemma sai hoặc đã đổi, sai kiểu, tactic không đóng được mục tiêu hay vượt ngân sách heartbeat. Cách phổ biến là thử lại nguyên prompt, bỏ qua thông tin quý nhất là thông báo lỗi của trình kiểm chứng. Nút thắt của bài toán nằm ở quy mô, lập lịch và chi phí:

kiểm chứng nguội một tệp mất 63 đến 107 giây và khoảng 3,5 GB RAM, số ứng viên tăng từ 10^2 lên 10^4, mỗi lần gọi LLM tốn token, và mọi lần thử phải được ghi lại để phát lại và kiểm toán.

1.2. Câu hỏi nghiên cứu

RQ1: thông báo lỗi có cấu trúc của Lean có cải thiện tỷ lệ sửa thành công so với thử lại nguyên prompt hay không, khi giữ nguyên dữ liệu, ngân sách, phần cứng và seed. RQ2: chính sách định tuyến và ngân sách thử lại nào cho tỷ lệ thành công tốt nhất dưới ngân sách token và thời gian. RQ3: hệ thống mở rộng thế nào theo số ứng viên 100, 1.000, 10.000 và số worker 1, 2, 4 về thông lượng, độ trễ P50, P95 và bộ nhớ.

1.3. Đóng góp

Báo cáo đóng góp bốn điểm: bộ dữ liệu lỗi Lean 4 v1.0 có nhãn sinh từ trình kiểm chứng và tập hold-out tách theo định lý gốc; taxonomy lỗi v0.2 đối chiếu với Lean 4.35, bổ sung nhóm TAC_07; pipeline sửa có giới hạn, có kiểm toán ngữ nghĩa và trần chi phí cứng; đánh giá đồng thời chất lượng, chi phí, thông lượng, độ trễ và bộ nhớ trên ba mức quy mô với ba lần lặp.

CHƯƠNG 2. CÔNG TRÌNH LIÊN QUAN

2.1. Môi trường và dữ liệu chứng minh

LeanDojo [1] cung cấp công cụ tương tác với Lean và cho thấy giá trị của truy hồi premise. Lean Workbook [2] hình thức hoá hàng chục nghìn bài toán thi đấu, ProofNet [3] là benchmark tự hình thức hoá cấp đại học. Nhóm dùng bộ chứng minh của Goedel-Prover [4] trên Lean Workbook làm nguồn định lý đúng.

2.2. Sinh và sửa chứng minh bằng LLM

Baldur [5] sinh toàn bộ chứng minh và sửa bằng cách đưa thông báo lỗi vào mô hình trên Isabelle/HOL, gần nhất với RQ1. Self-Debugging [6] cho thấy LLM tự sửa mã tốt hơn khi có phản hồi thực thi. DeepSeek-Prover-V1.5 [7] dùng phản hồi của Lean cho học tăng cường và tìm kiếm cây, DeepSeek-Prover-V2 [8] phân rã mục tiêu con, Draft-Sketch-Prove [9] dùng bản nháp phi hình thức để dẫn hướng.

2.3. Khoảng trống

Các công trình trên tối ưu chất lượng mô hình nhưng ít báo cáo đồng thời chi phí token, độ trễ đuôi và khả năng mở rộng của hệ thống kiểm chứng, là phần chiếm phần lớn thời gian thực tế. Nhóm coi verifier là tài nguyên khan hiếm cần lập lịch, theo tinh thần xử lý theo lô trong [10].

CHƯƠNG 3. DỮ LIỆU VÀ THU NHẬN

3.1. Nguồn và giấy phép

Định lý gốc lấy từ bộ Goedel-LM/Lean-workbook-proofs, giấy phép MIT, gồm 29.750 tệp Lean 4 hoàn chỉnh kèm đề bằng ngôn ngữ tự nhiên. Tệp nguồn được tải bằng script có kiểm tra SHA-256. Dữ liệu đầy đủ không đưa vào Git, chỉ lưu mẫu nhỏ và manifest ghi số dòng, checksum từng tệp.

3.2. Định lý gốc và lỗi tự nhiên

Nhóm lấy ngẫu nhiên có seed 5.000 chứng minh và kiểm chứng lại với Lean 4.35.0 và Mathlib v4.35.0, giới hạn heartbeat 400.000 và thời gian 30 giây. Kết quả có 3.666 định lý biên dịch sạch, chiếm 73,3%, được dùng làm seed. 1.110 chứng minh không còn biên dịch với Mathlib hiện tại, chủ yếu do đổi tên lemma như le_div_iff thành le_div_iff₀ hoặc cú pháp cũ, được giữ làm lỗi tự

nhiên. Thông lượng đạt 1,08 ứng viên mỗi giây trên 4 worker; P95 lên tới 25,8 giây do các chứng minh nlinarith nặng.

3.3. Sinh lỗi tổng hợp

Mỗi seed nhận tối đa 5 trong 9 toán tử gây lỗi ở Bảng 3.1, chọn bằng bộ sinh ngẫu nhiên tất định. Một đột biến chỉ được giữ khi Lean thật sự báo lỗi; 2.388 trên 18.309 đột biến bị loại vì không làm hỏng chứng minh. Nhãn luôn lấy từ thông báo Lean thật, không lấy từ toán tử.

Bảng 3.1: Các toán tử sinh lỗi tổng hợp.

Toán tử	**Nhóm dự kiến**	**Cách gây lỗi**
syn_drop_paren, syn_drop_assign	SYN_01	xoá một dấu đóng ngoặc hoặc dấu :=
imp_drop_import	IMP_02	xoá toàn bộ dòng import
id_lemma_typo, id_tactic_typo	ID_03	làm sai tên lemma hoặc tên tactic
typ_change_type	TYP_04	đổi kiểu một biến giữa $\mathbb{R}$ và $\mathbb{N}$
tac_weaken	TAC_07	thay tactic mạnh bằng tactic yếu hơn
tmo_low_heartbeats	TMO_05	hạ heartbeat xuống 500
sem_sorry	SEM_06	thay chứng minh bằng sorry

3.4. Chia tập và chất lượng nhãn

Dataset v1.0 có 17.031 mẫu duy nhất theo SHA-256, gồm 15.921 lỗi tổng hợp và 1.110 lỗi tự nhiên. Phân bố nhãn: SYN_01 4.590, ID_03 3.978, SEM_06 2.292, IMP_02 2.269, TAC_07 1.827, TMO_05 1.707, TYP_04 338, UNK_99 30. Tập được chia 70/30 theo băm của định lý gốc nên không định lý nào xuất hiện ở cả train và hold-out. Bốn mức lồng nhau gồm 100, 1.000, 10.000 và toàn bộ; hold-out của mức 1.000 có 284 mẫu, của mức 10.000 có 2.907 mẫu. Nhãn tái lập 17.031 trên 17.031 từ chẩn đoán đã lưu và khớp nhóm dự kiến ở 94,4% mẫu

tổng hợp. Tập 50 lỗi viết tay ở tuần 1 chỉ khớp 15 trên 50 với Lean thật, là lý do chuyển sang nhãn sinh từ trình kiểm chứng.

3.5. Hạn chế dữ liệu

Lỗi tổng hợp có thể khác phân bố lỗi do LLM sinh ra, bài toán thiên về bất đẳng thức thi đấu, hold-out mức 1.000 chỉ có 13 lỗi tự nhiên, và chưa có dữ liệu thật từ Nhóm 5.

CHƯƠNG 4. PHƯƠNG PHÁP VÀ KIẾN TRÚC

4.1. Tổng quan

Mỗi ứng viên đi qua hàng đợi, bộ phân loại lỗi, bộ định tuyến, luật hoặc LLM, worker Lean REPL và bộ kiểm toán. Mẫu thành công được ghi vào đầu ra bàn giao; mẫu thất bại được thử lại nếu còn ngân sách. Mọi bước đều ghi log phát lại được.

4.2. Bộ kiểm chứng REPL thường trú

Mỗi worker giữ một tiến trình Lean REPL đã nạp Mathlib một lần và kiểm từng ứng viên trong môi trường gốc nên các lần kiểm cô lập với nhau. Dòng import được kiểm tra tồn tại trong bản build để lỗi IMP_02 vẫn quan sát được. Kết quả khớp 74 trên 74 lần với Lean chạy nguội. Khi tactic chạy quá thời gian hoặc REPL chết, cả nhóm tiến trình gồm lake và tiến trình repl con bị huỷ rồi khởi động lại, và lần kiểm hỏng không được dùng để gán nhãn.

4.3. Taxonomy và bộ phân loại

Bảy nhóm lỗi ở Bảng 4.1 được so khớp theo thứ tự ưu tiên trên thông báo thật. IMP_02 còn được suy ra khi mã không có import nhưng dùng ký hiệu Mathlib. Lỗi chính là lỗi có vị trí sớm nhất sau khi loại lỗi hệ quả: unsolved goals

được Lean báo ở dòng khai báo nhưng là hệ quả của tactic hỏng bên dưới. Sửa điểm này nâng độ khớp nhãn từ 85,1% lên 95,0%.

Bảng 4.1: Taxonomy lỗi v0.2.

Mã	Nhóm lỗi	Định tuyến
SYN_01	cú pháp	luật
IMP_02	thiếu import	luật
ID_03	định danh không tồn tại hoặc đã đổi tên	lai
TYP_04	sai kiểu, thiếu instance	LLM
TMO_05	vượt heartbeat	điều khiển hệ thống
SEM_06	dùng sorry hoặc đổi đề	kiểm toán
TAC_07	tactic không đóng được mục tiêu	LLM

4.4. Luật sửa và sửa bằng LLM

Tám luật không tốn token gồm R0 khôi phục import Mathlib, R1 và R2 sửa ngoặc và từ khoá, R3 tra bảng import, R4 bí danh định danh, R5 bảng đổi tên Mathlib, R6 cú pháp toán tử lớn và R7 khôi phục heartbeat. LLM có hai chế độ dùng chung system prompt cấm sorry, axiom và yêu cầu giữ nguyên phát biểu. Chế độ blind chỉ đưa tệp lỗi, đúng baseline của đề bài. Chế độ chẩn đoán đưa mã đánh số dòng và tối đa năm thông báo Lean kèm nhãn taxonomy. Trước khi gửi, chú thích trong mã được thay bằng khoảng trắng để giữ nguyên dòng và cột, giảm 66% ký tự; 500 trên 500 seed đã xử lý vẫn biên dịch. Mô hình là Gemini 3.5 Flash-Lite với nhiệt độ 0, seed 42 và mức suy nghĩ tối thiểu.

4.5. Vòng lặp có giới hạn và kiểm toán

Mỗi mẫu được thử tối đa k lần trong 300 giây. Vòng lặp dừng khi thành công, hết ngân sách, không còn luật áp dụng hoặc ứng viên lặp lại theo SHA-256, nên không thể lặp vô hạn. Một bản sửa chỉ được tính là thành công khi Lean biên dịch, không dùng sorry, không chứa cấu trúc cấm như admit, axiom, native_decide, và phát biểu định lý trùng bản gốc sau khi chuẩn hoá khoảng

trắng. Cách so sánh văn bản này là bảo thủ: có thể đếm thiếu nhưng không đếm thừa.

4.6. Lập lịch, chi phí và độ phức tạp

Pool luồng gán mỗi worker một REPL, hàng đợi FIFO, đo riêng thời gian chờ và thời gian phục vụ. Trước mỗi lần gọi LLM, chi phí xấu nhất được kiểm với trần 8 USD mỗi tháng và 2 đến 3 USD mỗi lần chạy; vượt trần thì từ chối trước khi gọi. Bộ đệm câu trả lời giúp các lần lặp không tốn phí. Với N ứng viên, W worker, số lần thử trung bình $\bar{k}$, thời gian kiểm chứng v, độ trễ LLM ℓ và tỷ lệ lần thử đi qua LLM p, tổng thời gian xấp xỉ $T_0 + N \cdot \bar{k} \cdot (v + p \cdot \ell)/W$, trong đó T_0 là thời gian nạp Mathlib, từ 11 đến 25 giây mỗi worker trên Kaggle. Bộ nhớ tăng gần tuyến tính theo W và giới hạn số worker trên một máy.

CHƯƠNG 5. THIẾT LẬP THỰC NGHIỆM

5.1. Môi trường

Thực nghiệm chạy trên Kaggle CPU với Linux x86_64, 4 vCPU và 31,3 GB RAM, dùng Lean 4.35.0, Mathlib v4.35.0, Lean REPL v4.35.0 và Python 3.13. Máy phát triển Apple M1 16 GB chỉ dùng cho phát triển và kiểm thử, vì tại đây hai worker REPL đã đẩy swap lên hơn 14 GB.

5.2. Chiến lược và ma trận thí nghiệm

Năm chiến lược gồm không sửa, chỉ luật, blind là baseline bắt buộc, LLM có chẩn đoán, và lai với router. Mọi chiến lược dùng cùng tập hold-out, timeout Lean 60 giây, phần cứng và seed 42.

Bảng 5.1: Ma trận thí nghiệm; lần lặp 2 và 3 của cấu hình LLM dùng bộ đệm câu trả lời.

Nhóm	Dữ liệu	Cấu hình	Lặp
RQ1	hold-out 1K, 284 mẫu	blind k=1, LLM chẩn đoán k=1	3
RQ2	hold-out 1K	không sửa, chỉ luật k=3, LLM k=3, lai k=1 và 3	2 đến 3
RQ2 mô hình	hold-out 1K	LLM chẩn đoán k=1 với Gemini 3.6 Flash	1
RQ2 quy mô	hold-out 10K, 2.907 mẫu	lai k=1	1
RQ3	100, 1K, 10K	chỉ luật k=1 với 1, 2, 4 worker	3

5.3. Chỉ số và tái lập

Chỉ số chính là tỷ lệ sửa thành công đã kiểm toán cho RQ1, RQ2 và thông lượng ổn định cho RQ3. Chỉ số phụ gồm tỷ lệ biên dịch, số lần thử mỗi mẫu, chi phí USD định giá theo token, thời gian phục vụ P50 và P95, thời gian chờ và bộ nhớ đỉnh USS. Thông lượng ổn định loại trừ thời gian nạp Mathlib. Kết quả lặp được báo cáo bằng trung vị và khoảng min đến max. Mỗi lần chạy ghi một run record bất biến gồm commit, phiên bản dữ liệu kèm checksum, máy, phần mềm, tham số, seed, timeout, chỉ số và trạng thái, cùng log từng mẫu để phát lại. Mọi hình và bảng được sinh bằng script từ dữ liệu thô.

CHƯƠNG 6. KẾT QUẢ

6.1. Khởi động nguội là nút thắt đầu tiên

Trên máy phát triển, kiểm chứng một tệp bằng lake env lean mất 63 đến 107 giây vì phải nạp Mathlib. Với REPL thường trú, sau một lần nạp khoảng 107 giây, mỗi định lý chỉ mất 3 ms ở P50 và 23 ms ở P95, nhanh hơn ba đến bốn bậc

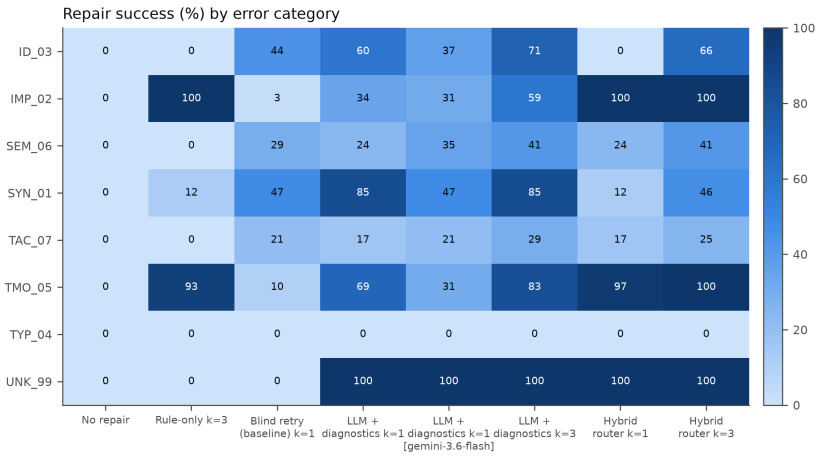
độ lớn. Trên Kaggle, mỗi worker nạp Mathlib trong 11 đến 25 giây và việc sinh dữ liệu đạt P50 70 ms, P95 2,1 giây mỗi mẫu.

6.2. RQ1: thông báo lỗi có cấu trúc

Bảng 6.1: RQ1 trên 284 mẫu hold-out, ba lần lặp, Gemini 3.5 Flash-Lite.

Chiến lược, k=1	**Thành công**	**Biên dịch**	**Token vào/ra mỗi lần gọi**	**USD/1.000 mẫu**
Blind, baseline	31,7%	34,9%	319 / 783	2,05
LLM có chẩn đoán	53,9%	59,2%	535 / 389	1,13

Tiêm chẩn đoán tăng 22,2 điểm phần trăm tỷ lệ thành công đã kiểm toán và giảm 45% chi phí. Prompt dài hơn 68% nhưng câu trả lời ngắn bằng một nửa, vì mô hình sửa cục bộ thay vì viết lại cả tệp, trong khi token đầu ra đắt gấp 8,3 lần token đầu vào. Lợi ích lớn nhất ở các lỗi có thông báo đặc hiệu: SYN_01 tăng từ 47% lên 85%, TMO_05 từ 10% lên 69%, IMP_02 từ 3% lên 34%, như ở Hình 6.1.



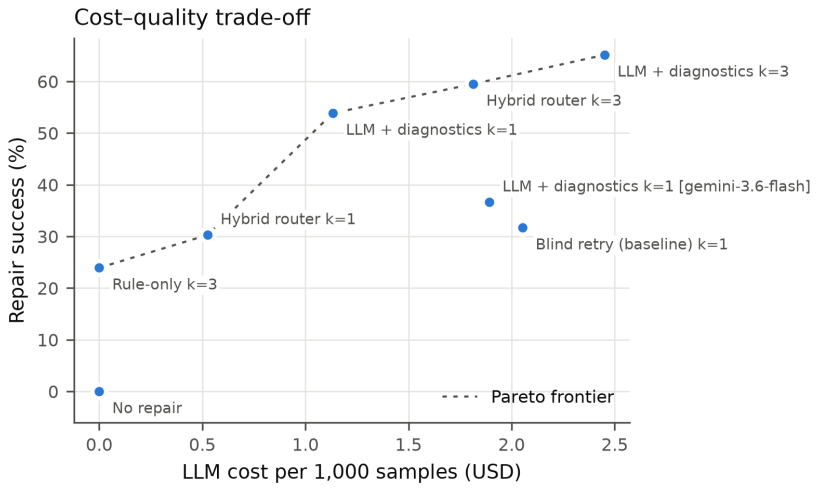
Hình 6: 1. Tỷ lệ sửa thành công theo nhóm lỗi và chiến lược trên hold-out 1K.

6.3. RQ2: định tuyến, ngân sách thử lại và mô hình

Bảng 6.2: Ablation trên hold-out 1K, trung vị hai đến ba lần lặp, chênh lệch giữa các lần không quá 0,7 điểm.

Chiến lược	k	Thành công	Biên dịch	Lần thử/mẫu	USD/1.000 mẫu	P95 phục vụ
Không sửa	0	0,0%	0,0%	0	0	không áp dụng
Chỉ luật	3	23,9%	24,6%	0,82	0	6,4 s
Lai	1	30,3%	32,8%	1,00	0,53	9,4 s
LLM chẩn đoán	1	53,9%	59,2%	1,00	1,13	16,4 s
Lai	3	59,5%	66,2%	2,14	1,81	23,3 s
LLM chẩn đoán	3	65,1%	75,0%	1,82	2,45	19,3 s
LLM, Gemini 3.6 Flash	1	36,6%	44,0%	1,00	1,89	18,9 s
Lai, hold-out 10K	1	34,1%	37,5%	1,00	0,34	7,7 s

Biên Pareto chi phí và chất lượng ở Hình 6.2 đi qua chỉ luật với 0 USD và 23,9%, lai k=1 với 0,53 USD và 30,3%, LLM k=1 với 1,13 USD và 53,9%, lai k=3 với 1,81 USD và 59,5%, rồi LLM k=3 với 2,45 USD và 65,1%. Có ba quan sát. Thứ nhất, ngân sách ba lần thử thêm 11,2 điểm với chi phí gấp 2,2 lần; 82,7% số ca thành công đến ngay lần thử đầu, 12,4% ở lần hai và 4,9% ở lần ba, nên lợi ích giảm dần. Thứ hai, lai kém LLM thuần ở cùng k vì luật được thử trước trên 203 trong 284 mẫu nhưng chỉ đúng 33%; khi luật sai và k=1 thì không còn lượt cho LLM. Thứ ba, mô hình lớn hơn không tốt hơn dưới cùng cấu hình: Gemini 3.6 Flash chỉ đạt 36,6%.



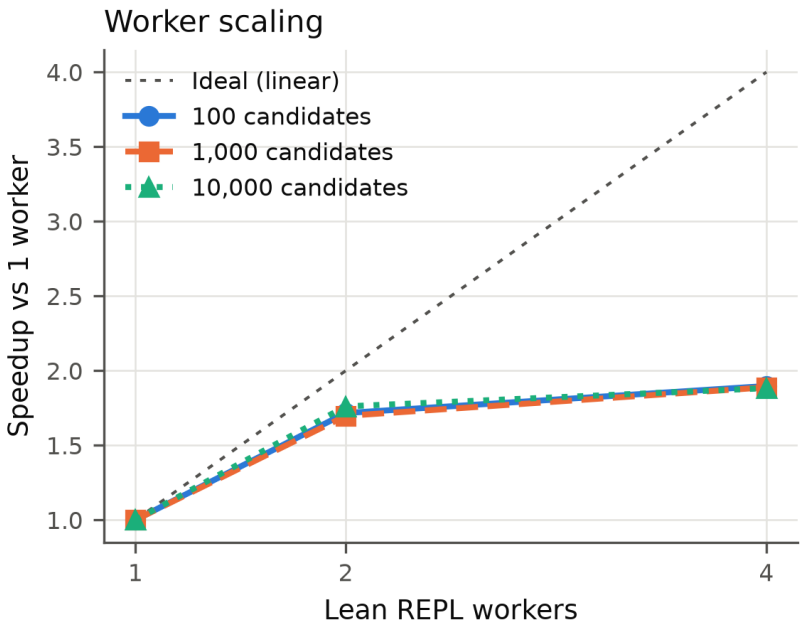
Hình 6: 2. Chi phí LLM cho 1.000 mẫu và tỷ lệ sửa thành công; đường chấm là biên Pareto.

6.4. RQ3: khả năng mở rộng

Bảng 6.3: Chỉ luật k=1 trên toàn bộ mỗi mức, ba lần lặp, chênh lệch thông lượng không quá 4%.

Mức	Worker	Mẫu/giây	Tăng tốc	P50 phục vụ	P95 phục vụ	USS đỉnh
100	1 / 2 / 4	0,95 / 1,62 / 1,79	1,00 / 1,72 / 1,90	66 / 73 / 128 ms	10,2 / 11,9 / 17,9 s	8,6 / 8,8 / 12,8 GB
1K	1 / 2 / 4	1,32 / 2,24 / 2,49	1,00 / 1,70 / 1,89	47 / 54 / 80 ms	4,3 / 4,7 / 8,8 s	9,2 / 16,1 / 16,3 GB
10K	1 / 2 / 4	1,52 / 2,68 / 2,86	1,00 / 1,76 / 1,88	51 / 57 / 93 ms	3,1 / 3,4 / 6,5 s	12,6 / 15,5 / 21,1 GB

Từ 1 lên 2 worker, thông lượng tăng gần tuyến tính 1,70 đến 1,76 lần; từ 2 lên 4 worker chỉ tăng thêm 7 đến 11% trong khi P95 thời gian phục vụ tăng gấp đôi và P50 tăng khoảng 65%, như ở Hình 6.3. Bốn tiến trình REPL tranh bốn vCPU với tiến trình điều phối và các tactic nặng, nên hệ thống bị giới hạn bởi CPU từ khoảng 2 worker. Tỷ lệ thành công không đổi theo số worker, 26,1% ở mức 10.000. Bộ nhớ được báo theo USS vì RSS cộng trùng phần Mathlib dùng chung, có lúc lên 35 GB trên máy 31 GB. P95 ở mức 100 cao hơn vì khi chỉ có 100 mẫu, P95 do khoảng 5 mẫu nặng nhất quyết định. Thời gian chờ trong chế độ chạy lô tăng tuyến tính theo số mẫu nên không dùng để so sánh giữa các mức.



Hình 6: 3. Tăng tốc thông lượng theo số worker ở ba mức dữ liệu so với đường tuyến tính lý tưởng.

CHƯƠNG 7. PHÂN TÍCH LỖI VÀ MỐI ĐE DOẠ TÍNH HỢP LỆ

7.1. Vì sao vòng lặp dừng và vì sao bản sửa bị loại

Bảng 7.1: Lý do dừng ở lần lặp đầu; cơ chế khử trùng lặp cắt 4,6% mẫu khỏi các lần thử vô ích.

Cấu hình	Thành công	Hết ngân sách	Không có luật	Ứng viên lặp
Blind k=1	32,0%	68,0%	0%	0%
LLM k=1	53,9%	46,1%	0%	0%
LLM k=3	65,1%	30,3%	0%	4,6%
Lai k=3	59,5%	38,7%	0%	1,8%
Chỉ luật k=3	23,9%	0%	76,1%	0%

Trong các mẫu thất bại, lần thử cuối bị loại chủ yếu vì không biên dịch, từ 77% đến 99% tuỳ cấu hình. Đáng chú ý, 17% đến 26% bản sửa của LLM tự ý đổi phát biểu và 10% đến 24% chèn sorry hoặc cấu trúc cấm. Những bản này có thể biên dịch được và chỉ bị chặn nhờ kiểm toán ngữ nghĩa. Không có kiểm toán, kết

quả sẽ bị thổi phồng: LLM k=3 biên dịch 75,0% nhưng thành công thật chỉ 65,1%.

7.2. Độ chính xác từng luật

Bảng 7.2: Độ chính xác của luật, gộp chỉ luật và lai trên hold-out 1K.

Luật	Số lần áp dụng	Qua kiểm toán	Độ chính xác
R3 bảng import	253	0	0%
R2 chèn từ khoá	126	0	0%
R0 khôi phục import	99	96	97%
R1 cân bằng ngoặc	99	27	27%
R7 khôi phục heartbeat	81	81	100%
R5, R4, R6	34	0	0%

Các luật viết cho đoạn mã ngắn ở tuần 4 không chuyển được sang tệp hoàn chỉnh: R2 chèn từ khoá làm mất định lý, R3 thêm import sai module, R1 nối ngoặc vào cuối tệp. Hai luật thiết kế từ lỗi thật là R0 và R7 gần như hoàn hảo. R5 đạt 0% vì lỗi tự nhiên thường gồm nhiều lỗi đan xen nên đổi tên một lemma chưa đủ. Từ đây, router chỉ nên gửi sang các luật có độ chính xác cao, đo trên tập train chứ không trên hold-out.

7.3. Theo nhóm lỗi, nguồn lỗi và mô hình

LLM k=3 sửa tốt SYN_01 với 85%, TMO_05 với 83%, ID_03 với 71%, nhưng kém ở TAC_07 với 29% và SEM_06 với 41%, hai nhóm đòi hỏi viết chứng minh mới. Lỗi tự nhiên khó hơn rõ rệt: LLM k=1 chỉ đạt 7,7% trên 13 mẫu so với 56,1% trên lỗi tổng hợp; với k=3 là 30,8% so với 66,8%. Gemini 3.6 Flash thua Flash-Lite chủ yếu vì nhiều lỗi biên dịch hơn, 125 so với 76 mẫu, và đổi phát biểu thường hơn. Cả hai chạy ở mức suy nghĩ tối thiểu, có thể bất lợi cho mô hình lớn, nên nhóm không kết luận mô hình lớn kém hơn.

7.4. Mối đe doạ tính hợp lệ

- Dữ liệu tổng hợp có thể dễ hoặc khó khác lỗi thật; lỗi tự nhiên được báo cáo tách riêng nhưng hold-out chỉ có 13 mẫu.
- Kiểm toán dạng văn bản tính phát biểu tương đương viết khác là đổi đề, nên có thể đếm thiếu.
- Lần lặp 2 và 3 của cấu hình LLM dùng bộ đệm, phản ánh tính tất định ở nhiệt độ 0 chứ không phản ánh biến thiên độ trễ API; chi phí được so sánh theo token, không theo chi tiêu thật.
- Kaggle là máy ảo dùng chung; mọi cấu hình của một mức chạy trong cùng phiên, xen kẽ lần lặp và báo cáo min đến max.
- Kết luận về LLM giới hạn ở họ Gemini Flash với mức suy nghĩ tối thiểu.
- Luật R0, R5, R6, R7 được thiết kế từ lỗi quan sát trên tập 50 mẫu cũ và seed thử nghiệm, không từ hold-out.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Với RQ1, tiêm chẩn đoán có cấu trúc của Lean tăng tỷ lệ sửa thành công đã kiểm toán từ 31,7% lên 53,9% và giảm 45% chi phí so với thử lại nguyên prompt. Với RQ2, ba lần thử với LLM có chẩn đoán đạt 65,1% ở 2,45 USD cho 1.000 mẫu, là điểm chất lượng cao nhất trên biên Pareto; chiến lược lai chỉ có lợi về chi phí và bị kéo xuống bởi các luật có độ chính xác thấp. Với RQ3, nút thắt đầu tiên là khởi động nguội Mathlib, được giải bằng REPL thường trú nhanh hơn ba đến bốn bậc độ lớn; nút thắt thứ hai là CPU, khiến thông lượng bão hoà từ khoảng 2 worker trên 4 vCPU. Kiểm toán ngữ nghĩa là bắt buộc vì 17% đến 26% bản sửa của LLM đổi đề hoặc lách bằng sorry.

Hướng phát triển gồm: router chọn luật theo độ chính xác đo trên tập train; hàng đợi ưu tiên cho lỗi rẻ và luật chính xác; phân tán worker qua nhiều máy vì một máy bão hoà CPU sớm; kiểm toán bằng so sánh kiểu đã elaborate thay cho

so sánh văn bản; dùng dữ liệu thật của Nhóm 5 và mô hình chuyên Lean mã nguồn mở chạy trên GPU.

TÀI LIỆU THAM KHẢO

[1] K. Yang et al., *LeanDojo: Theorem Proving with Retrieval-Augmented Language Models*, NeurIPS Datasets and Benchmarks, 2023.

[2] H. Ying et al., *Lean Workbook: A large-scale Lean problem set formalized from natural language math problems*, NeurIPS Datasets and Benchmarks, 2024.

[3] Z. Azerbayev et al., *ProofNet: Autoformalizing and Formally Proving Undergraduate-Level Mathematics*, arXiv:2302.12433, 2023.

[4] Y. Lin et al., *Goedel-Prover: A Frontier Model for Open-Source Automated Theorem Proving*, arXiv:2502.07640, 2025.

[5] E. First, M. Rabe, T. Ringer, Y. Brun, *Baldur: Whole-Proof Generation and Repair with Large Language Models*, ESEC/FSE, 2023.

[6] X. Chen, M. Lin, N. Schärli, D. Zhou, *Teaching Large Language Models to Self-Debug*, ICLR, 2024.

[7] H. Xin et al., *DeepSeek-Prover-V1.5: Harnessing Proof Assistant Feedback for Reinforcement Learning and Monte-Carlo Tree Search*, arXiv:2408.08152, 2024.

[8] Z. Z. Ren et al., *DeepSeek-Prover-V2: Advancing Formal Mathematical Reasoning via Reinforcement Learning for Subgoal Decomposition*, arXiv:2504.21801, 2025.

[9] A. Q. Jiang et al., *Draft, Sketch, and Prove: Guiding Formal Theorem Provers with Informal Proofs*, ICLR, 2023.

[10] J. Leskovec, A. Rajaraman, J. D. Ullman, *Mining of Massive Datasets*, Cambridge University Press.

PHỤ LỤC. SỬ DỤNG AI, PHÂN CÔNG VÀ TÁI LẬP

Nhóm sử dụng trợ lý AI Claude của Anthropic để hỗ trợ viết mã, thiết kế thực nghiệm, phân tích và soạn tài liệu. Gemini 3.5 Flash-Lite và 3.6 Flash là đối tượng nghiên cứu, không tham gia viết báo cáo. Mọi mã có AI hỗ trợ đều qua 96 kiểm thử tự động và chạy Lean thật, mọi số liệu truy được tới run record trong thư mục results/raw. Nhật ký sử dụng AI nằm ở docs/ai_use_log.jsonl và docs/ai_use_statement.md; phân vai, đóng góp đối chiếu lịch sử Git và phần bảo vệ của từng thành viên nằm ở contributions.md. Toàn bộ kết quả tái lập được bằng các lệnh trong README: cài đặt bằng scripts/kaggle_setup.sh, sinh dữ liệu bằng scripts/build_seeds.py và scripts/generate_error_dataset.py, chạy thực nghiệm bằng các notebook trong notebooks/sessions, và sinh hình bằng scripts/generate_figures.py.